SENIOR FULL-STACK ENGINEER · SEOUL, KR · F-2 VISA, NO SPONSORSHIP NEEDED

Benjamin Vasseur

I build production systems end to end: REST APIs and microservices in Node.js, plus the data-heavy React and WebGL interfaces on top. I've led a development team, run services on AWS and Kubernetes, and optimized demanding medical imaging tools.

60 FPS
UP FROM 3-4 FPS

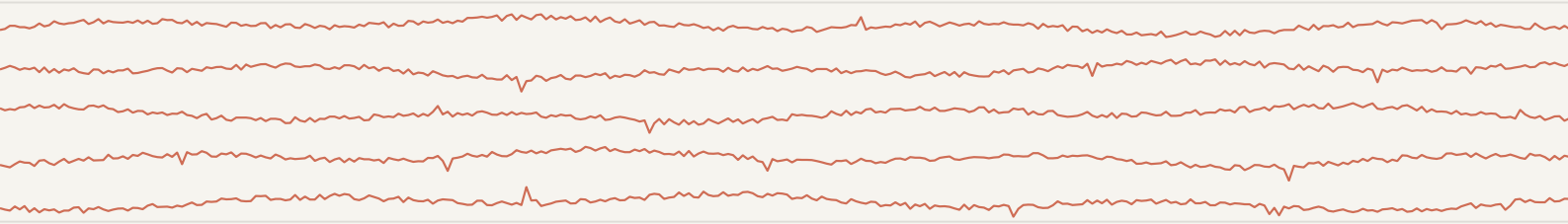
50%
LESS RENDER TIME

14+
YEARS EXPERIENCE

benvasseur59@gmail.com

LinkedIn ↗

Résumé ↗



ABOUT

I started in backend (PHP, then Node.js and Python) and still work across the full stack: REST APIs and microservices, the databases behind them, and deployment on AWS and Kubernetes. As lead at Global Coach Group, I owned technical direction for a platform serving Fortune 500 clients.

On the front end I've worked across Angular, Vue, and React. For 3+ years my work has centered on **high-performance rendering**: Canvas and WebGL for real-time, data-heavy interfaces in medical and scientific domains. More recently I've been building with LLM APIs. Based in Seoul, I work comfortably across international teams and time zones.

Live EEG Amplitude Heatmap

60 FPS

stable rendering,
up from 3–4 FPS

A clinical EEG platform streaming continuous multi-channel brain activity. The amplitude heatmap is the primary read surface: clinicians scrub through hours of signal, so any stutter directly hurts diagnosis.



Vue.js · Canvas API · TypeScript

CHALLENGE

The SVG implementation did full redraws on every update. Performance collapsed as live data streamed in, making real-time monitoring unusable.

SOLUTION

Rewrote the engine on Canvas with incremental drawing (only changed regions re-render) and trimmed the data pipeline to a stable frame rate.

RESULT

From 3–4 FPS to a steady 60 FPS. Smooth scrolling and real-time updates with zero frame drops during continuous streaming.

HIGHLIGHTS

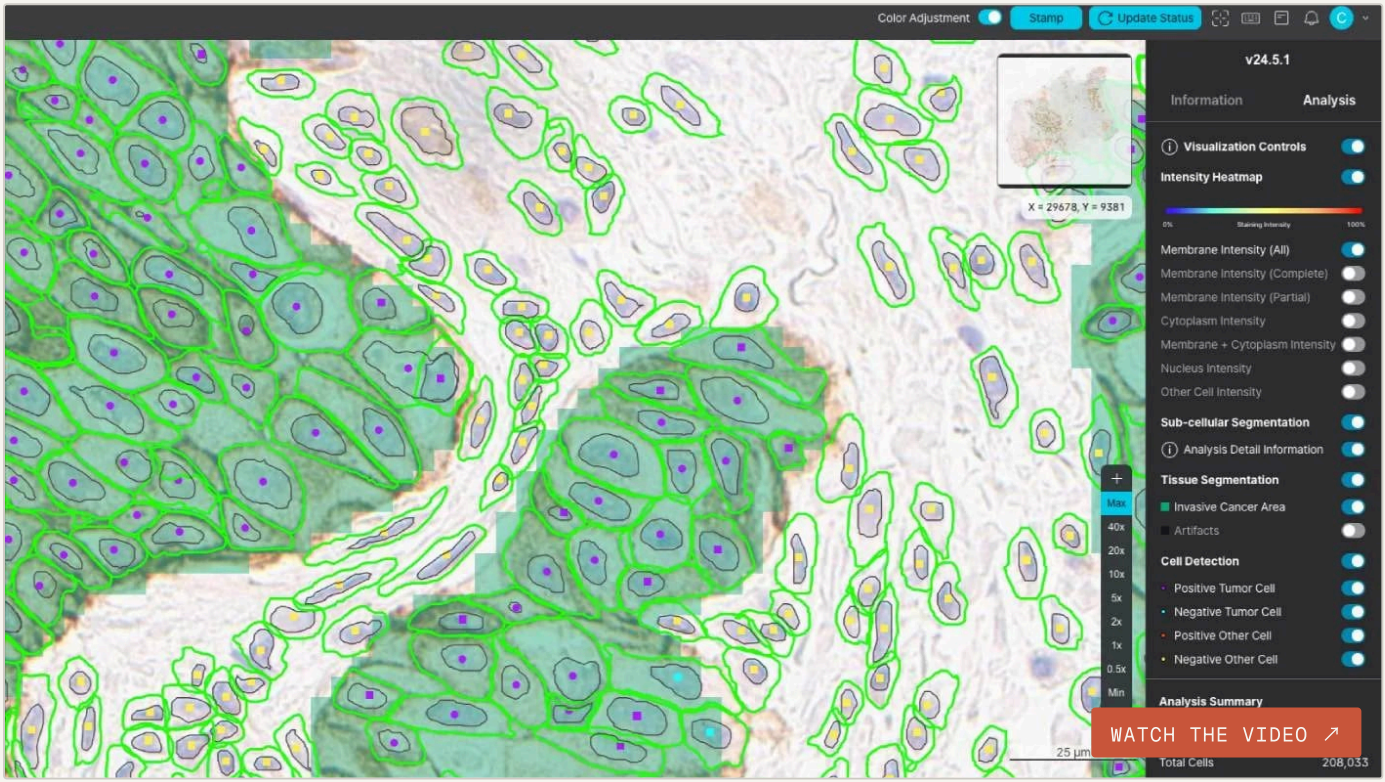
- Incremental region-based redraw instead of full-frame repaint
- Decoupled data ingestion from the render loop
- Held 60 FPS on hours-long continuous recordings

Cancer Cell Detection Viewer (uIHC v3)

50%

render time cut
on 5GB whole-slide images

A pathology viewer for AI-analyzed whole-slide images. Pathologists pan and zoom across gigapixel scans while millions of detected cells are drawn as an interactive overlay.



React · WebGL · Web Workers · TypeScript · Nx

CHALLENGE

Slides hold millions of cells with many properties. Zoom, pan, hover and threshold changes had to stay smooth on images up to 5GB.

SOLUTION

Capped minimum zoom to control density, moved heavy preprocessing to Web Workers, and used WebGL with tile-based loading for GPU-accelerated cells.

RESULT

Halved rendering time on million-cell slides while keeping all interactions smooth in real time.

HIGHLIGHTS

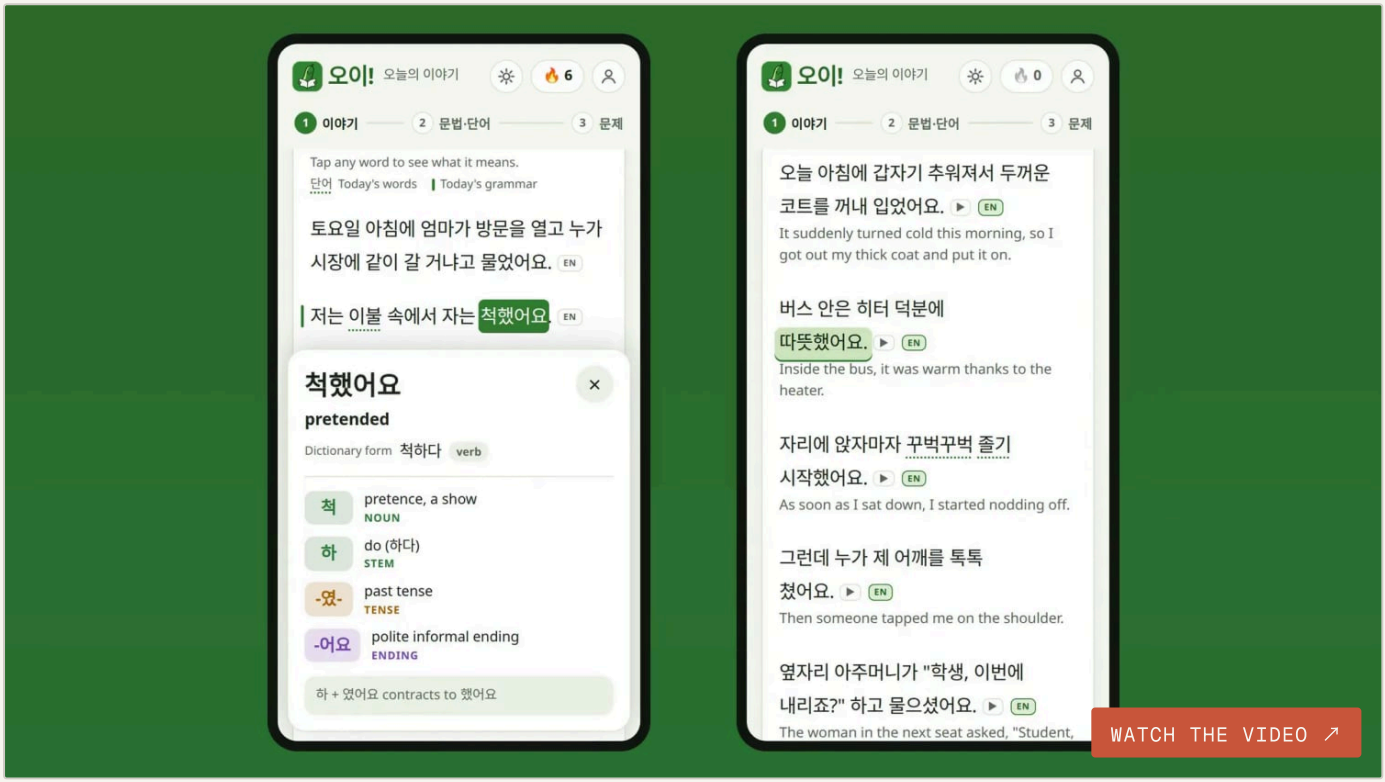
- GPU-instanced rendering of millions of cell markers
- Tile-based loading tied to the viewport
- Preprocessing offloaded to Web Workers to keep the main thread free

Daily Korean Reader (오늘의 이야기)

2+1

independent AI reviews,
then human approval

A daily reading habit for intermediate Korean learners (TOPIK 3–4): one short story a day with tap-to-gloss on every word, a grammar point, read-along audio and a TOPIK-style quiz. Reader accounts with Google and Kakao sign-in keep streaks and scores in sync.



Next.js · Payload CMS · PostgreSQL · Claude · OpenAI · whisper.cpp · Playwright

Visit oiyagi.com ↗

CHALLENGE

Publish a new, correct story every day as a solo developer. AI writes Korean at scale, but one unnatural sentence or wrong gloss loses learners, and native-level review doesn't scale.

SOLUTION

A model-agnostic pipeline: any AI can write a story as a pull request (Claude, Codex, Gemini and Mistral so far). Separate Claude and Codex sessions then review it independently and solve the quiz blind. CI merges only the exact verified content; a person approves and schedules every story in the Payload admin.

RESULT

Live since October 15 with a new story every day, AI-drawn covers and read-along audio where the spoken word lights up. 340+ automated tests across Vitest and Playwright.

HIGHLIGHTS

- Two independent AI reviews plus a blind quiz solve, tied to a content hash before CI can merge; Mistral was tested as a third reviewer and dropped for weak Korean
- Read-along audio: Google TTS voices word-timed by a local whisper.cpp, calibrated so 67 of 68 words land within 150 ms
- Payload 3 and Next.js in one app on Vercel and Supabase (Seoul); Google and Kakao OAuth in plain fetch, no auth library

~2s

click → finished
Anki card, end to end

Korean Anki Miner

A browser extension that turns any Korean word on a YouTube subtitle into a complete Anki flashcard: screenshot, translation and an AI-written explanation, in a couple of seconds.



Vue 3 · TypeScript · WXT · Chrome MV3 · Claude · Mistral

View on GitHub ↗

CHALLENGE

Read YouTube captions reliably across SPA navigation, isolate the overlay from YouTube's CSS, and keep API keys away from hostile page scripts.

SOLUTION

A strict trust boundary keeps secrets in the service worker; the UI mounts in a Shadow DOM. Papago handles fast lookups, Claude handles structured card enrichment.

RESULT

A releasable MV3 extension: click a word, get a finished card with screenshot and AI explanation in seconds. CI builds and publishes on every tagged release.

HIGHLIGHTS

- Secrets isolated in the MV3 service worker, never exposed to the page
- Shadow-DOM overlay immune to YouTube style bleed
- Papago for fast lookups, Claude for structured card enrichment

Experience

2011-Present

Sep 2025 - Present	**Independent** Full-Stack Engineer · Seoul	Korean study and self-directed products. Shipped 오늘의 이야기, a daily Korean reader with an AI content pipeline, and Korean Anki Miner. Building Mosida, a multilingual wedding invitation platform (NestJS, Next.js, Postgres). Experimenting with browser games for learning Korean (Phaser 4). Play the games ↗
Aug 2024 - Sep 2025	**LVIS Corp** Software Engineer · Seoul	EEG analysis platform with AI. Canvas amplitude heatmap, 3–4 → 60 FPS. Led user/team app with Stripe & KG Inicis.
Dec 2021 - Dec 2023	**Lunit** Front-End Developer · Seoul	AI cancer-cell viewer on whole-slide images. React + WebGL + Web Workers; 50% less render time on 5GB slides. Nx/Turborepo design system.
Feb 2019 - Dec 2021	**Global Coach Group** Full-Stack Lead · Seoul	Led a team of 3 on a platform for Fortune 500 clients. Node.js PDF-report microservice, Django/Node REST API design, AWS/Kubernetes. 13 languages via Lokalise.
Jun 2015 - Feb 2019	**3i Inc.** Full-Stack Developer · Seoul	SaaS for 3D/VR content authoring. Real-time collaboration over WebSockets; auth & role-based permissions.
May 2014 - May 2015	**Faire Savoir** Full-Stack Developer · Lille	Public-facing tourism sites on Drupal with booking & reservation features.
Sep 2011 - May 2014	**Élap** Developer · Lille	Financial software for the public sector. Expense/stock management, automated receipts & invoicing.

Skills

FRONT-END	TypeScript · React · Next.js · Vue.js · Vite · WebGL · Canvas · Tailwind
BACK-END	Node.js · Express · NestJS · Python · Django · Payload CMS · PHP · REST APIs · Microservices
AI	LLM API integration · Claude · OpenAI · Mistral AI
DATABASE	PostgreSQL · MySQL · MongoDB · MariaDB · Redis
DEVOPS	Docker · AWS · Kubernetes · CI/CD · Vercel · Cypress · Playwright · Git · Figma
LANGUAGES	French · native · English · fluent · Korean · TOPIK 2

OPEN TO NEW OPPORTUNITIES

Let's talk.

benvasseur59@gmail.com

LinkedIn ↗

Seoul, South Korea · F-2 visa, no sponsorship needed

FR native · EN fluent · KO TOPIK 2